

SALORIE

Catalogue de Prompts de Développement

Storytelling complet du développement de A à Z
+ 20 prompts futurs (améliorations & nouvelles features)

Ce document contient :

- Prompts de setup initial (projet, navigation, thème)
 - Prompts backend (Firebase, Clerk, Firestore)
- Prompts par feature (scan IA, analytics, i18n, etc.)
- Prompts d'intégrations tierces (Gemini, FatSecret, RevenueCat)
 - 1 prompt d'améliorations du projet actuel
 - 19 prompts de nouvelles features futures

Date : 20 avril 2026 — Version 1.0

Chaque prompt fait minimum 5 lignes et est directement exploitable.

Introduction

Ce document reconstitue l'intégralité du développement de l'application Salorie sous forme de **prompts réutilisables**. Chaque prompt est formulé comme une demande autonome et complète qu'un développeur (ou un agent IA) peut exécuter sans contexte préalable. L'ensemble est organisé chronologiquement : setup du projet → backend → authentification → features métier → intégrations tierces → optimisations.

Le catalogue se termine par **20 prompts prospectifs** : un premier prompt regroupe toutes les améliorations à apporter au projet actuel (refactor, bugs, dette technique), suivi de 19 prompts décrivant chacun une nouvelle feature potentielle pour la roadmap produit.

Structure du document

Partie	Contenu	Prompts
I	Setup initial & architecture	01 → 06
II	Backend & persistance	07 → 11
III	Authentification & onboarding	12 → 15
IV	Features métier (nutrition, IA)	16 → 28
V	Analytics & insights IA	29 → 32
VI	i18n, thème & UX transversale	33 → 36
VII	Intégrations tierces & monétisation	37 → 40
VIII	Testing, packaging & déploiement	41 → 43
IX	Améliorations du projet actuel	44
X	19 nouvelles features futures	45 → 63

Partie I — Setup initial & architecture

Prompt 01 — Setup : Initialisation du projet Expo SDK 52

Initialise un nouveau projet React Native avec Expo SDK 52 nommé **salorie** en TypeScript strict mode. Configure le package.json avec le script npm run start, active la New Architecture via newArchEnabled: true dans app.json, ajoute le bundle identifier Android com.idriss.kriouile.salorie, et active les experiments typedRoutes: true et reactCompiler: true pour bénéficier de la mémoïsation automatique de React Compiler 19 beta. Installe Expo Router v4 comme moteur de navigation principal avec un app/_layout.tsx racine.

Prompt 02 — Setup : Configuration TypeScript strict & ESLint

Configure un tsconfig.json strict avec strict: true, noImplicitAny, strictNullChecks et exactOptionalPropertyTypes. Ajoute les types Expo et React Native, puis crée un fichier ESLint v9 avec la config flat eslint.config.js héritant de expo/eslint-config. Définis des règles custom pour interdire les any implicites, forcer l'import typé avec type quand possible, et signaler les dépendances manquantes dans useEffect. Ajoute un script npm run lint.

Prompt 03 — Architecture : Structure de dossiers modulaire

Crée la structure de dossiers suivante : app/ pour les routes Expo Router (avec sous-groupes (tabs), (auth), (onboarding)), components/ pour les composants UI réutilisables, lib/ pour les services métier (Firebase, IA, i18n, contexts), hooks/ pour les hooks personnalisés, constants/ pour la configuration et les couleurs, scripts/ pour les utilitaires dev (seed), et assets/ pour les images et icônes. Documente chaque dossier avec un commentaire d'en-tête expliquant sa responsabilité.

Prompt 04 — Navigation : Expo Router avec typed routes et bottom tabs

Configure Expo Router v4 avec trois groupes logiques. Crée app/(tabs)/_layout.tsx qui définit un Tabs avec trois onglets : index (Home), analytics (Analytics) et profile (Profile), accompagnés d'icônes Lucide React Native (Home, BarChart3, User). Ajoute un Floating Action Button (FAB) central qui ouvre un bottom sheet ActionMenu. Configure le app/_layout.tsx racine pour brancher les providers (Clerk, Theme, i18n, Logging) dans le bon ordre et gérer la redirection entre auth / onboarding / tabs selon l'état de session.

Prompt 05 — Design System : Palette de couleurs & système de thème

Crée constants/Colors.ts exportant deux palettes (light et dark) avec : primary (vert #298f50 light / #4ade80 dark), secondary (ambre #f59e0b / #fbbf24), 9 nuances de gris, états success/warning/error, et un background principal. Implémente lib/ThemeContext.tsx qui expose {theme, setTheme, colors} avec trois modes : 'light', 'dark', 'system' (suit l'OS via useColorScheme()). Persiste le choix en AsyncStorage sous la clé app_theme et synchronise le StatusBar et la NavigationBar Android aux couleurs du thème actif.

Prompt 06 — Design System : Composants UI de base réutilisables

Crée les composants réutilisables suivants dans components/ : ScreenBackground (wrapper avec gradient ou couleur pleine), ScreenTopBar (header avec branding, sélecteur de langue, toggle thème, cloche notifications), AppBrand (logo Salorie + wordmark), HalfProgress (anneau circulaire animé générique prenant {value, max, color, size}). Tous les composants doivent consommer useTheme() et useI18n() pour la cohérence visuelle et linguistique, et supporter le RTL pour l'arabe.

Partie II — Backend & persistance

Prompt 07 — Backend : Initialisation Firebase Firestore

Crée lib/firebase.ts qui initialise le SDK Firebase v12 avec la configuration issue de constants/config.ts (apiKey, authDomain, projectId, storageBucket, messagingSenderId, appId). Exporte les instances db (Firestore) et storage (Cloud Storage). Implémente une fonction emailToDocId(email) qui normalise un email (trim, lowercase, remplacement de caractères spéciaux par '_') pour générer un identifiant de document stable. Cette stratégie d'email-keying permet de résister aux ré-inscriptions Clerk et de garder une seule source de vérité par utilisateur.

Prompt 08 — Backend : CRUD utilisateur & journalisation nutritionnelle

Étends lib/firebase.ts avec les fonctions suivantes : saveUserToFirestore(email, clerkId, profileData) qui crée ou merge un document users/{emailDocId} ; getUserProfile(email) qui retourne UserProfile | null ; addNutritionLog(email, log) qui écrit dans la sous-collection users/{docId}/logs avec un serverTimestamp ; getNutritionLogs(email, date) qui query par date locale YYYY-MM-DD ; deleteNutritionLog(email, logId) ; et updateWeight(email, kg) qui écrit dans weight_history. Chaque fonction inclut un try/catch et log les erreurs.

Prompt 09 — Backend : Interfaces TypeScript du modèle de données

Déclare dans lib/firebase.ts les interfaces TypeScript suivantes avec tous leurs champs optionnels bien typés. UserProfile : email, firstName, lastName, imageUrl, onboarded, gender, goal, workoutFrequency, birthdate, height {feet, inches}, weight, nutritionalPlan {dailyCalories, proteins, carbs, fats, waterIntake, advice: string[]}, language ('en'|'fr'|'ar'), pushToken, preferences, createdAt, updatedAt. NutritionLog : id, email, type ('meal'|'activity'|'water'), name, calories, protein, carbs, fat, serving, intensity, duration, date, timestamp. Exporte toutes les interfaces pour réutilisation à travers l'app.

Prompt 10 — Backend : Règles de sécurité Firestore

Rédige le fichier firestore.rules qui sécurise l'accès aux collections. users/{docId}/** : lecture et écriture autorisées uniquement si request.auth.token.email == resource.data.email, avec une exception pour la création initiale (create) où request.auth.token.email == request.resource.data.email. translations_cache/** : lecture publique pour partage entre utilisateurs, écriture réservée aux utilisateurs authentifiés. Ajoute la validation des champs obligatoires (email, type, date pour les logs) et teste avec l'émulateur Firestore avant déploiement.

Prompt 11 — Cache : Miroir AsyncStorage pour mode offline

Crée lib/LocalDataStore.ts qui maintient un miroir AsyncStorage de toutes les collections Firestore de l'utilisateur. Implémente syncAllUserData(email) qui télécharge profile, logs (30 derniers jours), weight_history, notifications et insights puis les persiste sous les clés profile_{docId}, logs_{docId}, etc. Ajoute updateLocalCollection(docId, key, data) pour les mises à jour partielles et readLocalCollection(docId, key) pour la lecture à froid. Implémente une queue de mutations offline qui se rejoue automatiquement à la reconnexion.

Partie III — Authentification & onboarding

Prompt 12 — Auth : Intégration Clerk (email/password + Google OAuth)

Installe @clerk/clerk-expo v2.19+ et configure le ClerkProvider dans app/_layout.tsx avec tokenCache basé sur Expo SecureStore. Crée les écrans app/(auth)/sign-in.tsx et app/(auth)/sign-up.tsx avec formulaires email/password validés côté client, plus un bouton Google SSO qui déclenche useOAuth({ strategy: 'oauth_google' }). Configure le warm browser pour une UX fluide et gère le callback via app/oauth-callback.tsx. À la connexion réussie, appelle saveUserToFirestore pour créer le document user.

Prompt 13 — Auth : Routing conditionnel selon l'état de session

Dans app/_layout.tsx, implémente une state machine de routing qui réagit à useAuth() de Clerk. Si !isLoading, affiche un splash screen. Si !isLoading, redirige vers /welcome. Si signé mais !profile.onboarded, redirige vers /(onboarding). Sinon vers /(tabs). Utilise un flag optimiste onboarded_{userId} en AsyncStorage pour accélérer le splash bypass au cold start (éviter le flicker). Gère proprement le deep-linking OAuth et les cas d'erreur réseau pendant le chargement.

Prompt 14 — Onboarding : Wizard 5 étapes personnalisation utilisateur

Crée app/(onboarding)/index.tsx, un wizard en 5 étapes avec barre de progression animée (Reanimated). Étape 1 : genre (male/female) avec grandes cartes visuelles. Étape 2 : goal (lose/gain/maintain weight) avec icônes explicites. Étape 3 : fréquence d'entraînement (sedentary/light/moderate/vigorous). Étape 4 : date de naissance avec trois pickers (jour, mois, année) et validation (âge minimum 13, pas de date future). Étape 5 : taille (feet/inches) et poids (kg) avec sliders ergonomiques. Chaque étape sauvegarde localement et permet la navigation arrière sans perte de données.

Prompt 15 — Onboarding : Génération du plan nutritionnel par IA Gemini

Crée app/(onboarding)/results.tsx qui, à l'arrivée de l'étape 5, envoie le profil complet à Gemini 2.5-flash via generateNutritionalPlan(profile) dans lib/AiModel.ts. Le prompt Gemini calcule le BMR avec la formule Mifflin-St Jeor, applique le facteur d'activité, ajuste selon le goal, et renvoie un JSON {dailyCalories, proteins, carbs, fats, waterIntake, advice[]}. Implémente un fallback offline déterministe qui calcule les mêmes valeurs localement si Gemini échoue (quota, timeout, JSON malformé). Affiche une animation de chargement et un résumé du plan avant de marquer onboarded: true et rediriger vers le home.

Partie IV — Features métier (nutrition & IA)

Prompt 16 — Feature : Dashboard Home avec anneaux de progression

Crée app/(tabs)/index.tsx, le dashboard principal. Il consomme useNutritionData(selectedDate) qui retourne {profile, logs, consumed, burned, water, loading}. Affiche : HomeHeader (salutation + avatar + date picker), CaloriesCard (anneau circulaire des calories consommées vs goal), WaterIntakeCard (anneau d'hydratation), RemainingCaloriesCard (kcal restantes avec couleur dynamique vert/jaune/rouge selon seuil), et ActivityList (FlatList des logs du jour avec swipe-to-delete). Intègre un WeekCalendar horizontal permettant de naviguer sur les 7 derniers jours.

Prompt 17 — Feature : LoggingContext pour state management global

Crée lib/LoggingContext.tsx, un Context React qui expose {selectedDate, setSelectedDate, logModalOpen, actionMenuOpen, refreshCount, triggerRefresh, capturedImageBase64, setCapturedImage}. Il sert de bus entre les écrans modaux (scan, recherche food, add water) et le dashboard. Le refreshCount est incrémenté à chaque mutation réussie pour invalider les caches des hooks qui l'observent. L'image base64 capturée est stockée ici plutôt qu'en URL params pour éviter la sérialisation et supporter les gros payloads. Wrap l'app dans ce provider au niveau root.

Prompt 18 — Feature : ActionMenu bottom sheet avec 4 méthodes de logging

Crée components/ActionMenu.tsx, un bottom sheet modal ouvert via le FAB. Il présente 4 grandes actions : **Scan Food** (ouvre un sélecteur caméra/galerie via expo-image-picker, demande les permissions, compresse l'image, stocke le base64 dans LoggingContext, navigue vers /scan-analysis) ; **Food Database** (navigue vers /food-database) ; **Exercise** (navigue vers /log-exercise) ; **Water** (navigue vers /add-water). Le sheet utilise Reanimated pour l'animation d'entrée/sortie et ferme sur tap backdrop.

Prompt 19 — Feature : Scan alimentaire par Gemini Vision

Crée app/scan-analysis.tsx qui récupère capturedImageBase64 du LoggingContext et l'envoie à Gemini Vision via un prompt structuré demandant d'identifier le plat, d'estimer la taille de portion, et de retourner un JSON strict {name, calories, protein, carbs, fat, serving}. Affiche un stepper de progression pendant l'appel (Uploading → Analyzing → Parsing). Valide la réponse field-by-field et permet à l'utilisateur d'ajuster la portion via un slider avant de confirmer. Au tap sur « Log Meal », appelle addNutritionLog puis markInsightsStale pour invalider les analytics, et ferme l'écran.

Prompt 20 — Feature : Recherche FatSecret avec OAuth2 & debounce

Crée lib/fatsecret.ts qui gère l'OAuth2 client-credentials avec FatSecret API v2 : getAccessToken() récupère un token, le cache en AsyncStorage avec son expiration, et le refresh automatiquement. searchFoods(query) appelle l'endpoint /foods/search/v3 et retourne un tableau normalisé. Crée ensuite app/food-database.tsx avec un TextInput debouncé à 500 ms, un état loading, et une FlatList de résultats. Au tap, navigue vers /log-food-details en passant l'aliment sélectionné en paramètre sérialisé.

Prompt 21 — Feature : Ajustement de portion avec recalcul proportionnel

Crée app/log-food-details.tsx qui reçoit un aliment (nom, calories, macros, serving de base) et permet à l'utilisateur d'ajuster la quantité. Affiche un input numérique et/ou un stepper +/- pour multiplier la portion (0.5x, 1x, 1.5x, 2x, custom). Recalcule en temps réel les calories et les 3 macros (protein, carbs, fat) proportionnellement. Affiche un résumé visuel avant confirmation. Au tap sur « Log Meal », écrit le log via addNutritionLog, déclenche triggerRefresh() du contexte, et retourne au home. Gère les cas d'erreur (input non numérique, quantité négative, valeurs extrêmes).

Prompt 22 — Feature : Logging manuel de repas & d'activités

Crée `components/LogModal.tsx`, un modal léger accessible depuis le home pour le logging rapide sans quitter le dashboard. Il propose un toggle entre trois modes : meal, activity, water. En mode meal : champs name, calories, protein, carbs, fat (tous optionnels sauf name et calories). En mode activity : name, calories burned, intensity (low/medium/high), duration. En mode water : sélecteur ml (125/250/500/750/1000). Validation côté client des champs obligatoires, prévention du double-tap sur submit, et feedback haptique (expo-haptics) à la confirmation pour l'UX.

Prompt 23 — Feature : Écran dédié d'exercice avec presets

Crée `app/log-exercise.tsx` qui présente une liste de presets d'activités (Running, Weight Lifting, Walking, Cycling, HIIT, Yoga) avec icônes Lucide. Au tap sur un preset, navigue vers `/workout-details` qui permet de paramétrer durée (minutes) et intensité. La formule d'estimation de calories brûlées prend en compte le poids de l'utilisateur (issu du profile), le MET de l'activité, et la durée. Après validation, navigue vers `/workout-result` qui affiche un résumé motivant (« Great job! You burned X calories ») avant de retourner au home avec log persisté.

Prompt 24 — Feature : Tracking d'hydratation avec UI glass picker

Crée `app/add-water.tsx` avec une UI de type « glass picker » : un grand verre animé qui se remplit visuellement selon le volume sélectionné (images empty/half/full). Boutons + et – pour incrémenter/décrémenter de 125 ml (half-glass). Limite maximale de 1000 ml par session (4 verres pleins), bouton + désactivé à la limite. Limite minimale de 0 ml, bouton – désactivé à 0. À la confirmation, appelle `addNutritionLog` avec type: 'water' et calories: waterMl (convention : le champ calories stocke les ml pour les logs de type water).

Prompt 25 — Feature : Édition des goals nutritionnels

Crée `app/personal-details.tsx` accessible depuis le profile. Il affiche le nutritionalPlan courant (dailyCalories, proteins, carbs, fats, waterIntake) sous forme de champs numériques éditables. Permet à l'utilisateur de remplacer les valeurs générées par Gemini par des goals personnels. À la sauvegarde, merge dans le document users via `updateUserProfile` et propage via `triggerRefresh()` pour que le home affiche immédiatement les nouveaux seuils. Ajoute un bouton « Reset to AI-generated » qui restaure le plan initial. Gère la validation : valeurs positives, somme des macros cohérente avec le total calorique.

Prompt 26 — Feature : Tracking du poids corporel

Crée `app/update-weight.tsx` qui permet de logger le poids courant (kg). L'écran affiche le dernier poids enregistré, un input avec stepper 0.1 kg, et un bouton « Save ». À la confirmation, écrit dans `users/{docId}/weight_history` avec la date du jour et le timestamp, et met à jour le champ weight sur le document user principal (dernière valeur connue). Expose également `getWeightHistory(email, days)` dans `lib/firebase.ts` pour alimenter un LineChart de tendance dans l'écran Analytics.

Prompt 27 — Feature : Swipe-to-delete & édition inline des logs

Étends `components/ActivityList.tsx` pour supporter le swipe horizontal (react-native-gesture-handler) qui révèle un bouton « Delete » rouge. Au tap delete, confirmation via Alert natif, puis `deleteNutritionLog(email, logId)` et `triggerRefresh()`. Ajoute aussi le tap court sur un log qui ouvre un modal d'édition avec les champs pré-remplis. À la sauvegarde, appelle `updateDoc` sur le log et flippe le flag stale des insights via `markInsightsStale`. Gère proprement les états d'erreur (permission denied, réseau coupé) avec toast UI.

Prompt 28 — Feature : Navigation entre dates & retour à aujourd'hui

Étends `HomeHeader` pour inclure un `DateTimePicker` natif iOS/Android qui permet de naviguer vers n'importe quelle date passée (blocage futur). Ajoute un bouton flottant « Today » qui apparaît dès que `selectedDate` n'est pas aujourd'hui et remet la sélection sur today au tap. Synchronise `selectedDate` avec `LoggingContext` pour que tous les écrans (add-water, log-exercise, etc.) loggent automatiquement sur la date sélectionnée. Gère le changement minuit (passage 23:59 → 00:00) en détectant le rollover et en

resetant sur le nouveau jour.

Partie V — Analytics & insights IA

Prompt 29 — Analytics : Hook useAnalyticsData avec cache-first

Crée `hooks/useAnalyticsData.ts` qui, pour un email et un scope ('week'|'month'|'all'), calcule les bornes de dates, query `users/{docId}/logs` et agrège par jour : consumed (somme kcal des meals), burned (somme kcal des activités), water (somme ml). Retourne `{dates[], consumed[], burned[], water[], loading, error}`. Implémente une stratégie cache-first : lecture instantanée depuis `AsyncStorage` sous la clé `analytics_{docId}_{scope}`, puis re-fetch en background et diff pour détecter les changements. Invalide le cache quand `refreshCount` change.

Prompt 30 — Analytics : InsightsService avec TTL 7 jours & stale flag

Crée `lib/InsightsService.ts` qui orchestre les insights IA scopés par période. Implémente `buildPeriodKey(scope)` pour générer des clés déterministes (`week_YYYY-Www` via ISO week, `month_YYYY-MM`, `all_time`). Expose `getInsights({email, scope, profile, logs, force, onCacheHit})` qui lit d'abord `AsyncStorage` (paint immédiat via `onCacheHit`), puis compare avec `Firestore`, puis régénère via Gemini si nécessaire. Implémente `markInsightsStale(email)` qui flippe `stale: true` sur `week + month + all_time` docs lors de chaque nouveau log. TTL dur à 7 jours via `insights_synced_{docId}`.

Prompt 31 — Analytics : Génération Gemini multilingue en un seul appel

Dans `lib/AiModel.ts`, crée `generateMultilangBentoInsights(profile, logs, periodLabel)` qui construit un prompt Gemini 2.5-flash demandant de retourner un JSON unique contenant les 3 langues simultanément : `{healthScore, en: {summary, topFood, hydrationStatus, recommendation, exerciseAnalysis}, fr: {...}, ar: {...}}`. Cette approche divise par 3 le nombre d'appels IA et garantit la cohérence des chiffres entre langues. Ajoute une validation field-by-field : si un champ est vide dans une langue, patche-le depuis `buildOfflineMultilangInsight` qui calcule les mêmes valeurs à partir des logs réels. Tag source: 'ai' ou 'computed'.

Prompt 32 — Analytics : Écran Analytics avec Bento cards & charts

Crée `app/(tabs)/analytics.tsx` avec un toggle en haut (Week / Month / All Time). Affiche un `BarChart` hebdo (`react-native-chart-kit`) avec deux séries : consumed (barres vertes) et burned (barres oranges), plus une ligne horizontale pour le goal calorique. En dessous, une grille Bento 2×3 présentant le healthScore (grand), summary, topFood, hydrationStatus, recommendation, exerciseAnalysis. Chaque card consomme `pickLang(storedInsight, currentLang)` pour afficher le texte localisé sans appel Gemini supplémentaire au changement de langue. Ajoute un compteur de streak animé et un `LineChart` de tendance de poids si l'historique existe.

Partie VI — i18n, thème & UX transversale

Prompt 33 — i18n : Provider i18n avec dictionnaire EN/FR/AR

Crée lib/i18n.tsx, un Context React qui expose {lang, setLang, t, isRTL, dictionary}. Le dictionnaire statique contient 500+ clés organisées en namespaces (common, auth, onboarding, home, meal, activity, water, analytics, profile, notifications, errors). Chaque clé a sa valeur en EN, FR, AR. La fonction t(key, params?) récupère la traduction et supporte les placeholders {{name}}. Persiste le choix en AsyncStorage et synchronise I18nManager.forceRTL(isRTL) pour l'arabe. Ajoute un hook utilitaire useI18n() qui retourne le context.

Prompt 34 — i18n : Traduction runtime à 4 couches de cache

Crée lib/translator.ts qui expose translate(text, targetLang, localLookup?). Stratégie en 4 couches (cheap → expensive) : (1) dictionnaire local i18n via localLookup si fourni ; (2) AsyncStorage cache sous tx_{djb2Hash(lang:text)} ; (3) Firestore cache partagé translations_cache/{hash} (write-through) ; (4) fallback Gemini 2.5-flash avec prompt « Translate ... return ONLY the translation ». À chaque hit distant, write-through les couches supérieures. Utilisé pour traduire dynamiquement les noms d'aliments FatSecret et les résumés Gemini en cas de fallback.

Prompt 35 — UX : Sélecteur de langue dans la top bar & préférences

Étends ScreenTopBar avec un IconButton « Globe » qui ouvre un petit popover listant les 3 langues disponibles (■■ English, ■■ Français, ■■ ■■■■■■■■) avec un checkmark sur la langue active. Au tap, appelle setLang et recharge l'application via Updates.reloadAsync() si un toggle RTL est nécessaire (EN/FR → AR ou inverse). Duplique l'option dans app/preferences.tsx pour les utilisateurs qui ne pensent pas à chercher dans la top bar. Persiste également la langue sur le document user Firestore pour la retrouver au login sur un nouvel appareil.

Prompt 36 — UX : Gestion complète du RTL pour l'arabe

Audite toute l'application pour le support RTL. Remplace tous les marginLeft/marginRight par marginStart/marginEnd, paddingLeft/paddingRight par paddingStart/paddingEnd, et textAlign: 'left' par 'auto'. Inverse automatiquement les icônes directionnelles (ChevronLeft/Right, back arrows) via { transform: [{ scaleX: isRTL ? -1 : 1 }] }. Teste les FlatList horizontales (week calendar), les bottom sheets (ActionMenu), les charts (labels d'axe), et les formulaires (onboarding). Ajoute un test manuel QA dédié RTL dans la checklist de release.

Partie VII — Intégrations tierces & monétisation

Prompt 37 — Intégration : Expo Notifications & rappels programmés

Crée lib/NotificationService.ts. Implémente registerForPushNotifications() qui demande les permissions, récupère l'expoPushToken via Notifications.getExpoPushTokenAsync (skip sur Expo Go / simulateur), et le persiste sur le document user Firestore. Configure un canal Android 'default' avec priorité HIGH et vibration. Crée scheduleMealReminders() qui programme 4 rappels quotidiens : Breakfast 8h, Lunch 13h, Dinner 19h, Encouragement 11h. Les messages sont traduits dans la langue de l'utilisateur via translate() et peuvent être personnalisés via un document admin Firestore.

Prompt 38 — Intégration : Historique des notifications & écran dédié

Étends NotificationService avec un listener Notifications.addNotificationReceivedListener qui appelle saveNotificationToHistory(email, {title, body, timestamp}). Cette fonction écrit dans users/{docId}/notifications et dans AsyncStorage pour consultation offline. Crée app/notifications.tsx qui liste l'historique en FlatList virtualisée (50 plus récentes), avec un indicateur « read/unread » et un pull-to-refresh. Ajoute un toggle dans app/preferences.tsx pour activer/désactiver globalement les notifications, qui appelle Notifications.cancelAllScheduledNotificationsAsync.

Prompt 39 — Monétisation : Intégration RevenueCat pour le Premium

Crée lib/PurchasesService.ts qui initialise le SDK react-native-purchases au mount de l'app avec les clés Android/iOS issues de CONFIG. Expose isPremium() qui appelle Purchases.getCustomerInfo() et teste entitlements.active['Premium']. Expose showPaywall() qui ouvre le paywall natif RevenueCat (configuré via le dashboard). Dans app/(tabs)/profile.tsx, affiche conditionnellement un bouton « Upgrade to Premium » qui appelle showPaywall. Après achat, rafraîchis le statut et débloquent les features premium (analytics all-time illimité, export CSV, coach IA étendu).

Prompt 40 — Intégration : Seed data pour tests & démo investisseurs

Crée scripts/seed-data.ts qui expose seedDemoData(email). La fonction nettoie d'abord les logs du jour pour éviter les doublons, puis génère pour les 11 derniers jours (aujourd'hui + 10 passés) un volume variable de meals (2 à 5), 1-2 activités, et un apport eau aléatoire. Les presets couvrent 12 meals (oatmeal, salad, pasta, smoothie...) et 6 exercices (run, lifting, yoga...). À la fin, écrit 3 documents ai_insights (week, month, all_time) avec du contenu riche EN/FR/AR tagué source: 'ai' pour que l'écran analytics soit immédiatement peuplé. Accessible depuis le bouton « Seed Demo Data » du profile.

Partie VIII — Testing, packaging & déploiement

Prompt 41 — OA : Plan de tests manuels par catégorie

Rédige une checklist QA couvrant 145 scénarios de tests regroupés en 14 catégories : authentification (12), onboarding (14), dashboard (13), logging de repas 4 méthodes (24), activités (12), hydratation (8), analytics (16), multilingue (12), profile & settings (14), notifications (10), mode offline (12), cas limites (14), sécurité (10), performance (10). Chaque scénario comprend un ID, un titre, des préconditions, des étapes numérotées et un résultat attendu. La checklist doit être exécutée manuellement sur un device physique Android et iOS avant chaque release majeure, avec un focus particulier sur le RTL arabe et les flux hors-ligne.

Prompt 42 — Build : Configuration EAS Build pour Android & iOS

Configure eas.json avec trois profils : development (APK development client), preview (APK distribution interne), production (AAB signé pour Play Store, IPA pour App Store). Configure les EAS Secrets pour toutes les variables EXPO_PUBLIC_* (Clerk, Gemini, Firebase ×6, RevenueCat ×2, FatSecret ×2). Ajoute le fichier google-services.json et GoogleService-Info.plist via EAS Secrets files. Définis le versionCode Android et le buildNumber iOS auto-incrémentés. Ajoute eas build --profile production --platform all au CI.

Prompt 43 — Déploiement : Checklist de release & rollout progressif

Rédige une checklist de release couvrant : (1) toutes les variables d'env configurées en EAS Secrets ; (2) règles Firestore publiées et testées avec l'émulateur ; (3) Clerk configuré avec le bon redirect URI OAuth pour l'app scheme production ; (4) paywalls RevenueCat publiés sur le dashboard avec les bons entitlements ; (5) icônes adaptive Android et splash iOS configurés ; (6) versionCode/buildNumber incrémentés ; (7) les 14 catégories de tests QA exécutées sur device physique ; (8) notifications testées sur device réel (pas Expo Go) ; (9) release notes rédigées en EN/FR/AR ; (10) rollout progressif Play Store (5 % → 25 % → 100 %) avec monitoring Crashlytics.

Partie IX — Améliorations du projet actuel

Prompt 44 — Refactor : Pack d'améliorations du codebase existant

Audit et refactor du codebase Salorie actuel selon les priorités suivantes. **(1) Dette technique** : découper lib/firebase.ts (504 lignes) en modules thématiques (firebase/client.ts, firebase/users.ts, firebase/logs.ts, firebase/insights.ts) ; extraire les types vers lib/types.ts pour éviter les imports circulaires. **(2) Invalidation cache** : corriger le bug connu où seed-data.ts écrit les insights sur Firestore sans invalider l'AsyncStorage local, ce qui cache le contenu fraîchement seedé derrière le cache précédent — ajouter AsyncStorage.removeItem(cacheKey(docId, periodKey)) pour les 3 scopes après le seed. **(3) Tests automatisés** : introduire Jest + React Native Testing Library, viser 60 % de couverture sur lib/ avec des tests unitaires (buildPeriodKey, emailToDocId, isEmpty, pickLang) et snapshot sur les cards principales. **(4) Performance** : mémoïser les composants lourds (CaloriesCard, ActivityList) avec React.memo ; remplacer les ScrollView par FlatList pour les listes > 20 items. **(5) Accessibilité** : ajouter accessibilityLabel, accessibilityRole et accessibilityHint sur tous les boutons ; vérifier les contrastes avec une cible WCAG AA. **(6) i18n** : extraire les strings en dur restants (notifications, erreurs Gemini, toast UI) dans le dictionnaire i18n ; auditer les clés non utilisées. **(7) Observabilité** : intégrer Sentry pour le crash reporting avec sourcemaps uploadés au CI, et logger les métriques clés (durée appel Gemini, taille cache, taux de cache hit) via un wrapper track(event, props). **(8) Sécurité** : ajouter des Firestore Rules de validation stricte (types, plages numériques, longueurs de string) ; rotate les clés API dans EAS Secrets. **(9) UX** : ajouter des skeletons de chargement cohérents sur home/analytics ; pull-to-refresh partout ; gestion explicite des états vides avec illustrations. **(10) CI/CD** : mettre en place GitHub Actions qui lint + typecheck + test à chaque PR, et déclenche un EAS build preview automatique sur merge vers main. Livrer le refactor en 10 PRs atomiques pour faciliter la review.

Partie X — 19 nouvelles features futures

Les 19 prompts suivants décrivent des features candidates pour la roadmap 2026-2027. Chacun est formulé comme une demande autonome prête à être priorisée en sprint planning.

Prompt 45 — Feature : Coach IA conversationnel (chatbot santé)

Implémente un onglet « Coach » qui ouvre un chatbot conversationnel propulsé par Gemini 2.5-flash en mode streaming. Le coach a accès aux 30 derniers jours de logs, au profile, au nutritionalPlan et à l'historique de poids pour répondre contextuellement (« Pourquoi j'ai pas perdu de poids cette semaine ? », « Que manger après mon workout ? »). Implémente un system prompt strict qui limite les sujets à la nutrition/fitness et refuse les demandes médicales (redirection vers professionnel). Stocke les conversations dans `users/{docId}/conversations/{convId}/messages` avec pagination, support du multilingue, et un disclaimer visible en permanence.

Prompt 46 — Feature : Scan de code-barres alimentaire

Ajoute un bouton « Scan Barcode » dans l'ActionMenu qui ouvre expo-camera en mode scan EAN-13/UPC-A. Au scan réussi, interroge l'API Open Food Facts (gratuit, 3M+ produits) avec le code-barres pour récupérer nom, macros, image, ingrédients, Nutri-Score. En fallback, interroge FatSecret par code-barres. Pré-remplit l'écran log-food-details avec les données récupérées. Cache localement les 500 produits les plus récents dans AsyncStorage pour lookup offline. Ajoute une UX de feedback haptique et visuel au scan, et un cadre de guidage pour aligner le code-barres.

Prompt 47 — Feature : Journal d'humeur & corrélations bien-être

Introduis un logging quotidien d'humeur sur une échelle de 5 (très mauvais → excellent) avec des tags contextuels (stressé, énergique, fatigué, motivé...). L'entrée se fait via un mini-modal déclenché à 20h par notification, ou manuellement depuis le home. Dans Analytics, ajoute une section « Wellness » qui corrèle via Gemini les patterns entre humeur et macros (ex. « Ton humeur est 30 % meilleure les jours où tu atteins ton goal protéines »). Respecte strictement la confidentialité : les données restent sur `users/{docId}/mood_logs` et ne sont jamais partagées ni utilisées pour l'entraînement d'un modèle tiers.

Prompt 48 — Feature : Plans de repas hebdomadaires générés par IA

Crée une feature « Meal Plan » accessible depuis le profile. L'utilisateur configure ses préférences (végétarien, halal, sans gluten, nombre de repas/jour, budget, temps de préparation max). Gemini génère un plan sur 7 jours avec 3-5 repas/jour, chacun avec recette détaillée, liste d'ingrédients quantifiée, temps de prep, macros calculées. Sauvegarde le plan dans `users/{docId}/meal_plans/{planId}`. Ajoute un bouton « Log this meal » sur chaque recette qui pré-remplit un log nutritionnel. Export PDF du plan en 1 tap pour l'imprimer. Régénération possible si le plan ne plaît pas.

Prompt 49 — Feature : Liste de courses intelligente

À partir d'un meal plan actif ou d'une sélection manuelle de recettes, génère automatiquement une liste de courses agrégée par catégorie (légumes, protéines, féculents, laitages, épicerie). Additionne les quantités quand un même ingrédient apparaît plusieurs fois dans la semaine. Permet de cocher les items achetés (persisté localement), d'ajouter manuellement des items libres, et de partager la liste par SMS/WhatsApp via expo-sharing. Bonus : intégration avec les APIs Carrefour/Instacart (selon marché) pour commander directement en ligne, avec lien deeplink vers les apps partenaires.

Prompt 50 — Feature : Challenges communautaires & leaderboard

Introduis un système de challenges hebdomadaires/mensuels (« 30 jours sans sucre ajouté », « 10 000 pas/jour pendant 2 semaines », « 3 L d'eau/jour »). L'utilisateur s'inscrit à un challenge, ses logs alimentent automatiquement son score, et un leaderboard public (opt-in) affiche les top 100 participants avec pseudo et photo. Stockage dans `challenges/{challengeId}/participants/{userId}`. Ajoute un système de badges

débloqués à la complétion. Respecte la confidentialité : l'opt-in leaderboard est explicite, les métriques individuelles ne sont jamais exposées.

Prompt 51 — Feature : Suivi d'objectifs SMART avec milestones

Permet à l'utilisateur de définir un objectif long-terme SMART (ex. « Perdre 5 kg en 3 mois », « Tenir 150 g protéines/jour pendant 30 jours »). Le système calcule automatiquement des milestones intermédiaires et trace la trajectoire attendue vs réelle dans un LineChart. Notifications de célébration à chaque milestone atteint (25 %, 50 %, 75 %, 100 %) avec animations Lottie. Gemini génère un plan d'action ajusté chaque semaine selon la progression réelle (« Tu es en retard sur ton poids, réduis de 150 kcal/jour »). Stockage dans `users/{docId}/goals/{goalId}`.

Prompt 52 — Feature : Export CSV/PDF & partage médical

Ajoute une fonction export dans le profile : CSV de tous les logs (date, type, nom, kcal, macros), PDF de rapport mensuel avec graphiques, ou partage direct à un médecin/nutri par email avec un lien signé à expiration 7 jours. Le PDF inclut : résumé du mois, BarChart hebdomadaire, top 10 aliments, insights IA en français, historique de poids. Génération côté client via expo-print et partage via expo-sharing. Cette feature est particulièrement utile pour les utilisateurs en suivi médical (diabète, obésité) et justifie le tier Premium.

Prompt 53 — Feature : Intégration Apple Health & Google Fit

Synchronise automatiquement les données depuis Apple HealthKit (iOS) et Google Fit (Android) via react-native-health et react-native-google-fit. Importe : pas quotidiens, distance, calories brûlées par activité, fréquence cardiaque, poids (si loggé dans ces apps), sommeil. Merge avec les logs manuels Salorie en évitant les doublons (détection par timestamp proche ± 5 min). Ajoute un toggle dans preferences.tsx pour activer/désactiver la sync. Respecte les permissions OS et affiche un écran de onboarding expliquant pourquoi chaque permission est demandée.

Prompt 54 — Feature : Mode Famille / Comptes enfants supervisés

Permet à un compte parent de créer et superviser jusqu'à 4 sous-comptes enfants (goals adaptés par âge, interface simplifiée, pas d'accès au paywall). Le parent voit un dashboard consolidé de toute la famille et reçoit un rapport hebdomadaire (« Marie a bu 80 % de son goal eau cette semaine »). Stockage : `users/{parentDocId}/family/{childId}` avec règles Firestore dédiées. L'interface enfant désactive le scan caméra (privacy), les achats in-app, et les conversations IA, ne laissant que le logging manuel simplifié via grandes icônes tactiles.

Prompt 55 — Feature : Widget home screen iOS / Android

Implémente un widget home screen qui affiche en temps réel : calories consommées / goal, anneau eau, 3 derniers repas loggés. Utilise expo-widgets (ou react-native-widget-extension pour iOS 17+) avec refresh toutes les 15 minutes. Deep link au tap pour ouvrir l'app directement sur l'action correspondante (tap sur anneau eau → `/add-water`). Le widget respecte le thème système (dark/light) et affiche les données de l'utilisateur actif sans requérir la session (lecture depuis AsyncStorage seulement, pas d'appel Firebase pour économiser la batterie).

Prompt 56 — Feature : Reconnaissance vocale pour logging mains-libres

Ajoute un bouton micro sur le home et dans l'ActionMenu qui active expo-speech-recognition. L'utilisateur dicte son repas (« J'ai mangé une pizza margherita et bu deux verres d'eau ») et Gemini 2.5-flash parse la transcription en logs structurés via un prompt few-shot. Affiche un écran de confirmation avec les logs détectés avant persistance. Utile en voiture, cuisine, gym. Supporte EN/FR/AR natifs des moteurs de reconnaissance. Indicateur visuel animé pendant l'écoute, feedback haptique à la fin, et fallback texte si la transcription échoue.

Prompt 57 — Feature : Rapports Apple Watch & wearOS

Porte l'app sur Apple Watch (Swift/WatchKit via Expo extension) et Wear OS (Kotlin/Jetpack Compose). Fonctionnalités minimales : anneau de progression calories du jour, logging rapide d'eau par bouton physique, affichage des 3 derniers logs, notification de rappel. Synchronisation bidirectionnelle avec l'app mobile via WatchConnectivity / MessageClient. Les complications watchface affichent le % de goal calorique et le goal eau. Utile pour le logging rapide pendant l'entraînement sans sortir le téléphone, et donne un positionnement premium sur le marché des apps santé.

Prompt 58 — Feature : Mode Jeûne intermittent avec timer

Implémente un mode « Fasting » dédié aux utilisateurs pratiquant le jeûne intermittent (16:8, 18:6, OMAD, 5:2). L'utilisateur choisit son protocole, le timer démarre au tap et affiche une grande UI en plein écran (heures restantes, phase métabolique avec explications vulgarisées : glycogène → cétose). Notifications à chaque phase clé. Pendant la fenêtre de jeûne, le logging de meal est bloqué (seul l'eau est autorisée). Historique des jeûnes dans `users/{docId}/fasts` avec analytics dédiées (durée moyenne, taux de complétion, corrélation avec poids). Respecte les disclaimers médicaux.

Prompt 59 — Feature : Gamification avec XP, niveaux & avatars

Transforme l'engagement en expérience ludique. Chaque action loggée donne des XP (meal = 10 XP, activity = 25 XP, water goal atteint = 50 XP, streak 7j = 200 XP). L'utilisateur monte en niveau (1-100) avec déblocage progressif d'avatars, skins d'anneaux, thèmes de carte exclusifs. Leaderboard amis optionnel. Stockage : `users/{docId}/gamification` avec {xp, level, unlockedItems[], currentAvatar}. Attention à ne pas encourager les comportements malsains (sur-logging pour XP) : cap quotidien d'XP, pas d'incitation à dépasser les goals caloriques. Side project : prévoir une modération des contenus communautaires.

Prompt 60 — Feature : Assistant shopping avec scan de ticket de caisse

Ajoute une feature « Scan Receipt » qui capture un ticket de caisse (photo), l'envoie à Gemini Vision avec un prompt OCR spécialisé qui extrait la liste des produits alimentaires, puis matche chaque ligne avec Open Food Facts ou FatSecret pour récupérer les macros. Présente un écran d'édition où l'utilisateur peut corriger les erreurs OCR, ajuster les quantités, et sélectionner les items à pré-charger dans son inventaire (`users/{docId}/pantry`). Le pantry alimente ensuite les suggestions du meal planner (« Tu as déjà des lentilles, voici 3 recettes pour les utiliser »).

Prompt 61 — Feature : Mode grossesse & allaitement personnalisé

Ajoute un mode spécialisé activable dans le profile pour les utilisatrices enceintes ou allaitantes. Les goals caloriques sont recalculés (+340 kcal au T2, +452 kcal au T3, +500 kcal en allaitement) et les recommandations Gemini intègrent les besoins spécifiques (folates, fer, calcium, oméga-3, iode). Affiche des alertes sur les aliments à éviter (alcool, fromages au lait cru, poissons à haute teneur en mercure, charcuterie crue) avec explications pédagogiques. Respecte strictement les disclaimers : les recommandations ne remplacent jamais un suivi médical prénatal, affiché en bannière permanente.

Prompt 62 — Feature : Traduction IA des recettes utilisateur

Permet à l'utilisateur de coller une URL de recette (blog, Marmiton, AllRecipes...) ou un texte libre, et l'app : (1) scrape / parse le contenu via Gemini, (2) le traduit dans les 3 langues supportées, (3) calcule les macros à partir des ingrédients, (4) sauvegarde dans `users/{docId}/saved_recipes`. L'utilisateur peut ensuite logger la recette en 1 tap sur plusieurs jours. Cache les recettes les plus populaires dans une collection globale `public_recipes` pour accélérer les requêtes répétées d'autres utilisateurs (respect RGPD : pas de données personnelles partagées, seulement le contenu public de la recette).

Prompt 63 — Feature : Mode Médical avec export vers dossier santé

Introduis un mode « Medical » destiné aux utilisateurs en suivi clinique (diabète type 1/2, maladies rénales, troubles du comportement alimentaire encadrés). Il ajoute des champs de logging étendus : glycémie avant/après repas, tension artérielle, médicaments pris, symptômes digestifs. Génère un rapport PDF

mensuel compatible IHE PCC (profil médical standardisé) exportable vers Apple Health Records et Google Health Connect. Chiffrement end-to-end des données médicales avec clé dérivée du PIN utilisateur (jamais stockée en clair côté serveur). Certification HIPAA/RGPD santé requise avant publication dans les stores sur ce mode.

— Fin du catalogue de prompts Salorie —

63 prompts au total : 43 de développement + 1 d'améliorations + 19 de nouvelles features